

AI ASSISTED CODING

ASSIGNMENT - 16.1

ROLL NO. 2303A51836

Task1–StudentInformationSystemSchema Task:
Design a database schema for a Student
Information System and
generate queries using AI.

Instructions:

- **Tables: Students, Courses, Enrollments.**
- **Define primary keys, foreign keys, and relationships.**
- **Generate queries:**
 - o **Insert a new student record.**
 - o **Fetch all courses enrolled by a student.**
 - o **Count number of students in each course.**

Expected Output:

- **SQL CREATE TABLE statements and queries for student–
course relationships.**

Prompt:

Create a simple Student Information System using Python and SQLite (sqlite3 library).

Requirements:

- Create three tables: Students, Courses, Enrollments
- Use proper primary keys and foreign keys
- Show relationship between students and courses using Enrollments table

Also write Python code to:

- Insert a new student record
- Fetch all courses enrolled by a specific student
- Count number of students in each course

Keep the codes simple and readable

Use sqlite3 and make sure everything runs in one file.

Add some sample data for testing.

Code:

```
import sqlite3
from datetime import datetime

# Create connection and cursor
conn=sqlite3.connect(':memory:')
cursor = conn.cursor()
```

```

# Create Tables
def create_tables():
    # Students Table
    cursor.execute('''
        CREATE TABLE Students (
            student_id INTEGER PRIMARY KEY AUTOINCREMENT, name
            TEXT NOT NULL,
            email TEXT UNIQUE NOT NULL,
            enrollment_date TEXT NOT NULL
        )
    ''')

    # Courses Table
    cursor.execute('''
        CREATE TABLE Courses (
            course_id INTEGER PRIMARY KEY AUTOINCREMENT,
            course_name TEXT NOT NULL,
            credits INTEGER NOT NULL
        )
    ''')

    # Enrollments Table (Junction table for Many-to-Many relationship)
    cursor.execute('''
        CREATE TABLE Enrollments (
            enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
            student_id INTEGER NOT NULL,
            course_id INTEGER NOT NULL,
            enrollment_date TEXT NOT NULL,
            FOREIGN KEY (student_id) REFERENCES Students (student_id),
            FOREIGN KEY (course_id) REFERENCES Courses (course_id),
            UNIQUE (student_id, course_id)
        )
    ''')

    conn.commit()
    print("Tables created successfully!\n")

# Insert sample data
def insert_sample_data():
    # Insert Students

```

```

students = [
    ('Alice Johnson', 'alice@example.com', '2024-01-15'),
    ('Bob Smith', 'bob@example.com', '2024-01-16'),
    ('CharlieBrown', 'charlie@example.com', '2024-01-17'),
]

cursor.executemany('INSERTINTOStudents(name,email,
enrollment_date) VALUES (?, ?, ?)', students)

#InsertCourses
courses = [
    ('Python Basics', 3),
    ('Data Structures', 4),
    ('Web Development', 3),
    ('Database Design', 4),
]

cursor.executemany('INSERTINTOCourses(course_name,credits)VALUES (?,
?)', courses)

#InsertEnrollments
enrollments = [
    (1, 1, '2024-01-15'), # Alice -> Python Basics
    (1, 2, '2024-01-15'), # Alice -> Data Structures
    (2, 1, '2024-01-16'), # Bob -> Python Basics
    (2, 3, '2024-01-16'), # Bob -> Web Development
    (3, 2, '2024-01-17'), # Charlie -> Data Structures
    (3, 4, '2024-01-17'), #Charlie -> Database Design
]

cursor.executemany('INSERTINTOEnrollments(student_id,course_id,
enrollment_date) VALUES (?, ?, ?)', enrollments)

conn.commit()
print("Sampledata inserted successfully!\n")

# Insert a new student
definsert_student(name,email):
    try:
        cursor.execute('INSERTINTOStudents(name,email,
enrollment_date) VALUES (?, ?, ?)',
                        (name, email, datetime.now().strftime('%Y-%m-%d')))
        conn.commit()

```

```

        print(f"Student '{name}' added successfully!\n")
    except sqlite3.IntegrityError:
        print(f"Error: Email '{email}' already exists!\n")

# Fetch all courses for a specific student
def get_student_courses(student_name):
    cursor.execute('''
        SELECT s.name, c.course_name, c.credits
        FROM Students s
        JOIN Enrollment s ON s.student_id = e.student_id JOIN
        Courses c ON e.course_id = c.course_id
        WHERE s.name = ?
    ''', (student_name,))

    results = cursor.fetchall()
    if results:
        print(f"Courses for {student_name}:")
        for name, course, credits in results:
            print(f"    - {course} ({credits} credits)")
    else:
        print(f"No courses found for {student_name}")
        print()

# Count students in each course
def count_students_per_course():
    cursor.execute('''
        SELECT c.course_name, COUNT(e.student_id) as student_count
        FROM Courses c
        LEFT JOIN Enrollment e ON c.course_id = e.course_id
        GROUP BY c.course_name
        ORDER BY student_count DESC
    ''')

    results = cursor.fetchall()
    print("Student Count Per Course:")
    for course, count in results:
        print(f"    - {course}: {count} students")
    print()

# Main execution

```

```
if name__ == 'main':  
    create_tables()  
    insert_sample_data()  
  
    # Insert a new student  
    insert_student('DianaPrince', 'diana@example.com')  
  
    # Get courses for specific students  
    get_student_courses('AliceJohnson')  
    get_student_courses('Bob Smith')  
  
    #Countstudentspercourse  
    count_students_per_course()  
  
    #Closeconnection  
    conn.close()
```

Output:

Tables created successfully!

Sampledatainsertedsuccessfully!

Student'DianaPrince'addedsuccessfully!

CoursesforAliceJohnson:

- PythonBasics(3credits)
- DataStructures(4credits)

CoursesforBobSmith:

- PythonBasics(3credits)

-WebDevelopment(3credits)

StudentCountPerCourse:

- PythonBasics:2students**
- DataStructures:2students**
- WebDevelopment:1students**
- DatabaseDesign:1students**

Task2–HospitalManagementDatabase

Task:

CreateschemaandqueriesforaHospital Management System.

Instructions:

- Tables:Doctors,Patients,Appointments.**
- UseAltodefineconstraints(uniqueIDs,valid dates).**
- Generatequeries:**
 - o Listallappointmentsforaspecificdoctor.**
 - o RetrievepatienthistorybypatientID.**
 - o Counttotalpatientstreatedbyeachdoctor.**

Expected Output:

- NormalizedschemaandSQLquerieswithjoins.**

Prompt:

Code:

```
import sqlite3
from datetime import datetime

# Connect to SQLite database
conn=sqlite3.connect(':memory:')
cursor = conn.cursor()

# Create tables
cursor.execute('''
    CREATE TABLE Doctors (
        doctor_idINTEGERPRIMARYKEYAUTOINCREMENT,
        name TEXT NOT NULL UNIQUE,
        specialization TEXT NOT NULL
    )
''')

cursor.execute('''
    CREATE TABLE Patients (
        patient_idINTEGERPRIMARYKEYAUTOINCREMENT, name
        TEXT NOT NULL,
        age INTEGER NOT NULL,
        email TEXT UNIQUE
    )
''')

cursor.execute('''
    CREATE TABLE Appointments (
        appointment_idINTEGERPRIMARYKEYAUTOINCREMENT, doctor_id
        INTEGER NOT NULL,
        patient_id INTEGER NOT NULL,
        appointment_dateTEXTNOTNULL, reason
        TEXT,
        FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
        FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
    )
''')
```



```

    )
    '''

# Insert sample data
print("=== Inserting Sample Data ===\n")

# Insert doctors
cursor.execute("INSERTINTODOctors(name,specialization)VALUES(?,?)", ("Dr.
    Smith", "Cardiology"))
cursor.execute("INSERTINTODOctors(name,specialization)VALUES(?,?)", ("Dr.
    Johnson", "Orthopedics"))
cursor.execute("INSERTINTODOctors(name,specialization)VALUES(?,?)", ("Dr.
    Williams", "Neurology"))

# Insert patients
cursor.execute("INSERTINTOPatients(name,age,email)VALUES(?,?,?)", ("Rohit", 25,
    "rohit@email.com"))
cursor.execute("INSERTINTOPatients(name,age,email)VALUES(?,?,?)", ("Priya", 30,
    "priya@email.com"))
cursor.execute("INSERTINTOPatients(name,age,email)VALUES(?,?,?)", ("Amit", 45,
    "amit@email.com"))
cursor.execute("INSERTINTOPatients(name,age,email)VALUES(?,?,?)", ("Neha", 28,
    "neha@email.com"))

# Insert appointments
cursor.execute("INSERTINTOAppointments(doctor_id,patient_id,
appointment_date, reason) VALUES (?, ?, ?, ?)",
    (1, 1, "2024-01-15", "Heart checkup"))
cursor.execute("INSERTINTOAppointments(doctor_id,patient_id,
appointment_date, reason) VALUES (?, ?, ?, ?)",
    (1, 2, "2024-01-16", "Chest pain"))
cursor.execute("INSERTINTOAppointments(doctor_id,patient_id,
appointment_date, reason) VALUES (?, ?, ?, ?)",
    (2, 3, "2024-01-17", "Knee pain"))
cursor.execute("INSERTINTOAppointments(doctor_id,patient_id,
appointment_date, reason) VALUES (?, ?, ?, ?)",
    (1, 4, "2024-01-18", "Regular checkup"))
cursor.execute("INSERTINTOAppointments(doctor_id,patient_id,
appointment_date, reason) VALUES (?, ?, ?, ?)",

```

```

        (3, 2, "2024-01-19", "Headache"))

conn.commit()
print("Data inserted successfully!\n")

#Query1:Listallappointmentsforaspecificdoctor print("===
Appointments for Dr. Smith ===")
doctor_name="Dr.Smith"cursor
.execute('''
    SELECTa.appointment_id,d.nameasdoctor_name,p.nameaspatient_name,
        a.appointment_date, a.reason
    FROM Appointments a
    JOIN Doctors d ON a.doctor_id = d.doctor_id
    JOINPatientspONa.patient_id=p.patient_id WHERE
    d.name = ?
    ORDERBYa.appointment_date
''', (doctor_name,))

for row in cursor.fetchall():
    print(f"ID:{row[0]},Doctor:{row[1]},Patient:{row[2]},Date:{row[3]}, Reason:
{row[4]}")
print()

#Query2:RetrievefullpatienthistoryusingpatientID print("===
Patient History for Rohit (ID: 1) ===")
patient_id = 1
cursor.execute('''
    SELECTp.patient_id,p.name,p.age,p.email,d.nameasdoctor_name,
        a.appointment_date, a.reason
    FROM Patients p
    LEFTJOINAppointmentsaONp.patient_id=a.patient_id LEFT
    JOIN Doctors d ON a.doctor_id = d.doctor_id
    WHERE p.patient_id = ?
    ORDERBYa.appointment_date
''', (patient_id,))

patient_info=cursor.fetchall() if
patient_info:
    p = patient_info[0]

```

```

print(f"Patient:{p[1]},Age:{p[2]},Email:{p[3]}") print("Appointments:")
for row in patient_info: if
    row[4]:
        print(f" -Dr. {row[4]}, Date: {row[5]}, Reason: {row[6]}")
print()

#Query3:Count total patient treated by each doctor
print("===Total Patients Treated by Each Doctor===")
cursor.execute('''
    SELECT d.name,d.specialization,COUNT(DISTINCT a.patient_id) as total_patients
    FROM Doctors d
    LEFT JOIN Appointments a ON d.doctor_id=a.doctor_id GROUP BY
    d.doctor_id, d.name, d.specialization
    ORDER BY total_patients DESC
''')

for row in cursor.fetchall():
    print(f"{row[0]} ({row[1]}): {row[2]} patients")

conn.close()

```

Output:

```
===Inserting Sample Data=== Data
```

```
inserted successfully!
```

```
===Appointments for Dr.Smith ===
```

```

ID:1, Doctor:Dr.Smith, Patient:Rohit, Date:
2024-01-15, Reason: Heart checkup
ID:2, Doctor:Dr.Smith, Patient:Priya,

```

Date:2024-01-16,Reason:Chestpain
ID:4,Doctor:Dr.Smith,Patient:Neha,
Date:2024-01-18,Reason:Regularcheckup

===PatientHistoryforRohit(ID:1)=== Patient:
Rohit, Age: 25, Email: rohit@email.com
Appointments:
-Dr.Dr.Smith,Date:2024-01-15, Reason:
Heart checkup

===TotalPatientsTreatedbyEachDoctor
===

Dr. Smith (Cardiology): 3 patients
Dr.Johnson(Orthopedics):1patients Dr.
Williams (Neurology): 1 patients

Task3–LibraryDatabase

Task:

DesignschemaforaLibraryManagementSystem.

Instructions:

- Tables:Books,Members,Loans.
- UseAltosuggestindexingstrategyforfaster queries.

- **Generate queries:**
 - o **Retrieve all books currently issued.**
 - o **Find overdue books (loan date > 30 days).**
 - o **Count number of books loaned by each member.**

Expected Output:

- **Schema + SQL queries demonstrating joins and conditions.**

Prompt:

Create a simple Library Management System using Python and SQLite (sqlite3 library).

Requirements:

- Create three tables: Books, Members, Loans
- Use proper primary keys and foreign keys
- Keep schema normalized and simple

Also:

- Suggest an indexing strategy to improve query performance

Write Python code to:

- Insert sample data
- Retrieve all books currently issued
- Find overdue books (loan date more than 30 days old)

- Count number of books loaned by each member

Use SQL joins and conditions where required.

Keep codes simple and readable and print outputs.

Code:

```
import sqlite3
from datetime import datetime, timedelta

# Connect to SQLite database
conn=sqlite3.connect(':memory:')
cursor = conn.cursor()

# Create tables with proper schema
cursor.execute('''
    CREATE TABLE Books (
        book_id INTEGER PRIMARY KEY AUTOINCREMENT, title
        TEXT NOT NULL,
        author TEXT NOT NULL,
        isbn TEXT UNIQUE NOT NULL
    )
''')

cursor.execute('''
    CREATE TABLE Members (
        member_id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        email TEXT UNIQUE NOT NULL,
        join_date DATE NOT NULL
    )
''')

cursor.execute('''
    CREATE TABLE Loans (
```

```

        loan_id INTEGER PRIMARY KEY AUTOINCREMENT,
        book_id INTEGER NOT NULL,
        member_id INTEGER NOT NULL, loan_date
        DATE NOT NULL,
        return_date DATE,
        FOREIGN KEY (book_id) REFERENCES Books(book_id),
        FOREIGN KEY (member_id) REFERENCES Members(member_id)
    )
'''

# Create indexes for performance
cursor.execute('CREATE INDEX idx_loans_member_id ON Loans(member_id)')
cursor.execute('CREATE INDEX idx_loans_book_id ON Loans(book_id)')
cursor.execute('CREATE INDEX idx_loans_loan_date ON Loans(loan_date)')
cursor.execute('CREATE INDEX idx_members_email ON Members(email)')

# Insert sample data
books = [
    ('Python Crash Course', 'Eric Matthes', '978-1593275099'),
    ('Clean Code', 'Robert Martin', '978-0132350884'),
    ('Design Patterns', 'Gang of Four', '978-0201633610')
]
cursor.executemany('INSERT INTO Books(title, author, isbn) VALUES(?, ?, ?)', books)

members = [
    ('Alice Johnson', 'alice@example.com', '2023-01-15'), ('Bob
    Smith', 'bob@example.com', '2023-02-20'),
    ('Carol White', 'carol@example.com', '2023-03-10')
]
cursor.executemany('INSERT INTO Members (name, email, join_date) VALUES (?, ?,
?)', members)

# Insert loans (some overdue, some active) today =
datetime.now().date()
loans = [
    (1, 1, today - timedelta(days=45), None), # Overdue
    (2, 2, today - timedelta(days=20), None), # Active
    (3, 1, today - timedelta(days=35), None), # Overdue

```

```

        (1, 3, today - timedelta(days=5), None),    # Active
        (2, 1, today - timedelta(days=25), today - timedelta(days=2))    # Returned
    ]
    cursor.executemany(
        'INSERT INTO Loans (book_id, member_id, loan_date, return_date) VALUES (?, ?, ?,?)',
        loans
    )

    conn.commit()

#Query1:Retrieveallbookscurrentlyissued
print("=== BOOKS CURRENTLY ISSUED ===")
cursor.execute('''
    SELECTDISTINCTb.book_id,b.title,b.author
    FROM Books b
    JOINLoanslONb.book_id=l.book_id WHERE
    l.return_date IS NULL
''')
for row in cursor.fetchall():
    print(f"ID:{row[0]}, Title: {row[1]}, Author: {row[2]}")

# Query 2: Find overdue books
print("\n=== OVERDUE BOOKS ===")
cursor.execute('''
    SELECTl.loan_id,b.title,m.name,l.loan_date FROM
    Loans l
    JOIN Books b ON l.book_id = b.book_id
    JOIN Members m ON l.member_id = m.member_id
    WHERE l.return_date IS NULL AND julianday('now') - julianday(l.loan_date) >
30
''')
for row in cursor.fetchall():
    print(f"Loan ID: {row[0]}, Book: {row[1]}, Member: {row[2]}, Loan Date: {row[3]}")

# Query 3: Count books loaned by each member
print("\n=== BOOKS LOANED BY EACH MEMBER ===")
cursor.execute('''

```



```

SELECT m.member_id, m.name, COUNT(l.loan_id) as total_loans FROM
Members m
LEFT JOIN Loans l ON m.member_id = l.member_id GROUP BY
m.member_id
ORDER BY total_loans DESC
'''
)
for row in cursor.fetchall():
    print(f"Member:{row[1]}, Total Loans: {row[2]}")

conn.close()

```

Output:

===BOOKSCURRENTLYISSUED ===

ID:1, Title:PythonCrashCourse, Author: Eric Matthes

ID:2, Title:CleanCode, Author:Robert Martin

ID:3, Title:DesignPatterns, Author:Gang of Four

===OVERDUEBOOKS ===

LoanID:1, Book:PythonCrashCourse,
Member: Alice Johnson, Loan Date:
2026-02-06

LoanID:3, Book:DesignPatterns, Member: Alice
Johnson, Loan Date: 2026-02-16

===BOOKSLOANEDBYEACHMEMBER===

```
Member:AliceJohnson,TotalLoans:3
Member: Bob Smith, Total Loans: 1
Member: Carol White, Total Loans: 1
```

Task4—Real-TimeApplication:OnlineShopping Database

Scenario:

DesignadatabaseforanE-commerceplatform.

Requirements:

- **Tables:Users,Products,Orders,OrderDetails.**
- **Generatequeries to:**
 - o **Retrieveallordersbya user.**
 - o **Findthemostpurchasedproduct.**
 - o **Calculatetotalrevenueinagivenmonth.**
- **Alshouldalsosuggestnormalization improvements and queryoptimization.**

Expected Output:

- **Completeschemawithrelationships+SQL queries for Analytics.**

Prompt:

Create a simple E-commerce database using Python and SQLite (sqlite3 library).

Requirements:

- Create tables: Users, Products, Orders, OrderDetails**
- Use proper primary keys and foreign keys**
- Maintain relationships between tables**

Also:

- Suggest normalization improvements**
- Suggest query optimization techniques**

Write Python code to:

- Insert sample data**
- Retrieve all orders by a specific user**
- Find the most purchased product**
- Calculate total revenue in a given month**

Use joins and aggregation functions.

Keep the codes simple and print outputs clearly.

Code:

```
import sqlite3
from datetime import datetime
```

```
# Create connection
conn=sqlite3.connect(':memory:')
cursor = conn.cursor()

#Createtableswithproperschema
cursor.execute('''
    CREATE TABLE Users (
        user_idINTEGERPRIMARYKEYAUTOINCREMENT, name
        TEXT NOT NULL,
        email TEXT UNIQUE NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
''')

cursor.execute('''
    CREATE TABLE Products (
        product_idINTEGERPRIMARYKEYAUTOINCREMENT, name
        TEXT NOT NULL,
        price REAL NOT NULL,
        stock INTEGER DEFAULT 0
    )
''')

cursor.execute('''
    CREATE TABLE Orders (
        order_id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER NOT NULL,
        order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        total_amount REAL,
        FOREIGN KEY (user_id) REFERENCES Users(user_id)
    )
''')

cursor.execute('''
    CREATE TABLE OrderDetails (
        detail_id INTEGER PRIMARY KEY AUTOINCREMENT,
        order_id INTEGER NOT NULL,
        product_id INTEGER NOT NULL,
```

```

        quantity INTEGER NOT NULL, unit_price
        REAL NOT NULL,
        FOREIGN KEY (order_id) REFERENCES Orders(order_id),
        FOREIGN KEY (product_id) REFERENCES Products(product_id)
    )
'''

# Insert sample data
cursor.execute("INSERT INTO Users(name,email)VALUES('Alice', 'alice@example.com')")
cursor.execute("INSERT INTO Users(name,email)VALUES('Bob',
'bob@example.com')")

cursor.execute("INSERT INTO Products(name,price,stock)VALUES('Laptop',1000, 5)")
cursor.execute("INSERT INTO Products(name,price,stock)VALUES('Mouse',25, 50)")
cursor.execute("INSERT INTO Products(name,price,stock)VALUES('Keyboard',75, 30)")

cursor.execute("INSERT INTO Orders(user_id,order_date,total_amount)VALUES(1, '2024-
01-15', 1025)")
cursor.execute("INSERT INTO Orders(user_id,order_date,total_amount)VALUES(1, '2024-
02-20', 75)")
cursor.execute("INSERT INTO Orders(user_id,order_date,total_amount)VALUES(2, '2024-
01-10', 100)")

cursor.execute("INSERT INTO OrderDetails(order_id,product_id,quantity, unit_price)
VALUES (1, 1, 1, 1000)")
cursor.execute("INSERT INTO OrderDetails(order_id,product_id,quantity, unit_price)
VALUES (1, 2, 1, 25)")
cursor.execute("INSERT INTO OrderDetails(order_id,product_id,quantity, unit_price)
VALUES (2, 3, 1, 75)")
cursor.execute("INSERT INTO OrderDetails(order_id,product_id,quantity, unit_price)
VALUES (3, 2, 4, 25)")

conn.commit()

# Query 1: Get all orders by a specific user

```

```

print("===OrdersbyUser'Alice'===")
cursor.execute('''
    SELECT o.order_id,o.order_date,o.total_amount
    FROM Orders o
    JOIN Users u ON u.user_id=o.user_id WHERE
    u.user_id = 1
    ORDER BY o.order_date DESC
''')
for row in cursor.fetchall():
    print(f"OrderID: {row[0]}, Date: {row[1]}, Amount: ${row[2]}")

#Query2:Find the most purchased product
print("\n=== Most Purchased Product ===")
cursor.execute('''
    SELECT p.name,SUM(od.quantity) as total_qty
    FROM OrderDetails od
    JOIN Products p ON od.product_id=p.product_id GROUP
    BY p.product_id
    ORDER BY total_qty DESC
    LIMIT 1
''')
result = cursor.fetchone()
print(f"Product:{result[0]}, Total Quantity Sold: {result[1]}")

#Query3:Calculate total revenue in January 2024
print("\n=== Revenue for January 2024 ===")
cursor.execute('''
    SELECT SUM(o.total_amount) as total_revenue
    FROM Orders o
    WHERE strftime('%Y-%m', o.order_date) = '2024-01'
''')
revenue = cursor.fetchone()[0]
print(f"Total Revenue:${revenue}")

conn.close()

# Normalization improvements:
# 1. Separate address info into Address table
# 2. Create ProductCategories table for better organization

```

```
# 3. Add indexes on frequently queried columns (user_id, product_id, order_date)

# Query optimization:
#1.Createindexes:CREATEINDEXidx_user_idONOrders(user_id) # 2.
Use EXPLAIN QUERY PLAN to analyze performance
# 3. Denormalize sparingly for read-heavy operations
```

Output:

```
===OrdersbyUser'Alice'===
OrderID:2,Date:2024-02-20,Amount:
$75.0
OrderID:1,Date:2024-01-15,Amount:
$1025.0

=== Most Purchased Product ===
Product:Mouse,TotalQuantitySold:5

===RevenueforJanuary2024=== Total
Revenue: $1125.0
```

Task 5 – University Examination Database Design
adatabaseschemaforaUniversity Examination System and generateSQLqueriesusingAI.
Instructions:

- **Tables: Students, Subjects, Exams, Results.**
 - **Define primary keys, foreign keys, and referential integrity constraints.**
 - **Generate queries:**
 - o **Insert examination results for a student.**
 - o **Retrieve subject-wise marks for a student.**
 - o **Calculate average marks for each subject.**
- Expected Output:**
- **Normalized schema and SQL queries involving aggregation and joins.**

Prompt:

Create a simple University Examination System using Python and SQLite (sqlite3 library).

Requirements:

- Create tables: Students, Subjects, Exams, Results
- Define primary keys and foreign keys properly
- Maintain referential integrity

Write Python code to:

- Insert sample data
- Insert examination results for a student

- Retrievesubject-wisemarksforastudent
- Calculateaveragemarksforeachsubject

Use joins and aggregation functions.

Keepthecodesimpleandreadable,andprintoutputs.

Code:

```
import sqlite3

# Connect to database
conn=sqlite3.connect(':memory:')
cursor = conn.cursor()

# Create tables
cursor.execute('''CREATE TABLE Students (
    student_idINTEGERPRIMARYKEYAUTOINCREMENT, name
    TEXT NOT NULL,
    enrollment_number TEXT UNIQUE NOT NULL
)''')

cursor.execute('''CREATE TABLE Subjects (
    subject_idINTEGERPRIMARYKEYAUTOINCREMENT, name
    TEXT NOT NULL,
    code TEXT UNIQUE NOT NULL
)''')

cursor.execute('''CREATE TABLE Exams (
    exam_id INTEGER PRIMARY KEY AUTOINCREMENT,
    exam_nameTEXTNOTNULL, exam_date
    TEXT NOT NULL
)''')

cursor.execute('''CREATE TABLE Results (
    result_id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```

        student_id INTEGER NOT NULL,
        subject_id INTEGER NOT NULL,
        exam_id INTEGER NOT NULL,
        marks REAL NOT NULL,
        FOREIGNKEY(student_id) REFERENCES Students(student_id),
        FOREIGNKEY(subject_id) REFERENCES Subjects(subject_id),
        FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)
    )'''

# Insert sample data
students=[('RohitKumar','EN001'),('PriyaSingh','EN002'),('AmitPatel','EN003')]
cursor.executemany('INSERT INTO Students (name, enrollment_number) VALUES (?, ?)', students)

subjects=[('Mathematics','MATH101'),('Physics','PHY101'),('Chemistry','CHM101')]
cursor.executemany('INSERT INTO Subjects (name, code) VALUES (?, ?)', subjects)

exams = [('Midterm', '2024-02-15'), ('Final', '2024-05-20')]
cursor.executemany('INSERT INTO Exams (exam_name, exam_date) VALUES (?, ?)', exams)

# Insert results
results = [
    (1, 1, 1, 85), (1, 2, 1, 78), (1, 3, 1, 92),
    (1, 1, 2, 88), (1, 2, 2, 81), (1, 3, 2, 95),
    (2, 1, 1, 90), (2, 2, 1, 88), (2, 3, 1, 85),
]

cursor.executemany('INSERT INTO Results (student_id, subject_id, exam_id, marks) VALUES (?, ?, ?, ?)', results)

conn.commit()

# Retrieve subject-wise marks for student 1
print("===Subject-wise Marks for RohitKumar===")
cursor.execute('''
    SELECT s.name, e.exam_name, r.marks FROM
    Results r
''')

```

```

JOIN Students s ON r.student_id = s.student_id JOIN
Subjects s ON r.subject_id = s.subject_id
JOIN Examse ON r.exam_id = e.exam_id WHERE
st.student_id = 1
ORDER BY s.name, e.exam_name
'''
)
for row in cursor.fetchall():
    print(f"{row[0]} ({row[1]}): {row[2]}")

# Calculate average marks for each subject
print("\n===AverageMarksbySubject===")
cursor.execute('''
    SELECT s.name, ROUND(AVG(r.marks), 2) as average_marks FROM
    Results r
    JOIN Subjects s ON r.subject_id = s.subject_id GROUP
    BY s.subject_id, s.name
    ORDER BY average_marks DESC
''')
for row in cursor.fetchall():
    print(f"{row[0]}: {row[1]}")

conn.close()

```

Output:

```

===Subject-wiseMarksforRohitKumar===
Chemistry (Final): 95.0
Chemistry(Midterm):92.0
Mathematics(Final):88.0
Mathematics(Midterm):85.0
Physics(Final):81.0
Physics(Midterm):78.0

```

```
===AverageMarksbySubject=== Chemistry:  
90.67  
Mathematics:87.67  
Physics:82.33
```